

8 Rest a JPA

8. 1. Vytváranie a odstraňovanie zdrojov

Príklad 1: REST služba jedálne - varianta POST

Príklad 2: REST služba jedálne – varianta PUT

8.2 Zdroje so štrukturovanými dátami

Príklad 3: REST služba jedálne – varianta POST XML

Implementácia klienta v jazyku Java

8.3 Integrácia REST a JPA

Príklad 4: REST služba jedálne – varianta POST s ukladaním v databáze

Cvičenia

8 Rest a JPA

8. 1. Vytváranie a odstraňovanie zdrojov

REST-API poskytuje tiež metódy na vytváranie nových, resp. odstraňovanie existujúcich zdrojov. Pri návrhu služby umožňujúcej vytvárať zdroje je potrebné rozhodnúť, či URI nového zdroja definuje server alebo klient:

- Ak URI definuje server, klient posielajú reprezentáciu nového zdroja požiadavkou POST na URI kolekcie, do ktorej nový zdroj pridáva.
- Ak URI nového zdroja definuje sám klient, potom posielajú jeho reprezentáciu priamo na URI nového zdroja požiadavkou PUT.

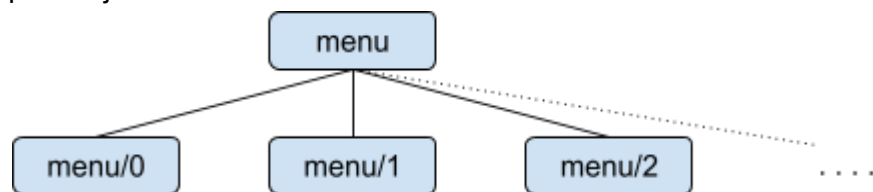
Rôzne varianty a ich implementácie si ilustrujeme na príklade webovej služby jedálne, ktorá bude poskytovať aktuálne informácie o ponuke jedál. Zároveň umožňuje vkladať a odstraňovať jedlá z ponuky.

Príklad 1: REST služba jedálne - varianta POST

Ponuka jedálne predstavuje kolekciu jedál a je prístupná ako koreňový zdroj s URI **menu**.

Jednotlivé

jedlá v ponuke dňa budú prístupné ako pod-zdroje s URI **menu/{n}**, kde n je poradové číslo jedla v ponuke, pričom jedlá sú číslované za sebou od 0.



Pre URI ponuky dňa **menu** poskytuje služba metódy:

- **GET** pre MIME `text/plain`: vráti aktuálny počet jedál v menu.
- **POST** pre MIME `text/plain`: pridá nové jedlo do kolekcie menu.

Pre URI jedla **menu/{n}** poskytuje služba metódy:

- **GET** pre MIME `text/plain`: vráti názov jedla s poradovým číslom n.
- **DELETE**: odstráni jedlo s poradovým číslom n.

Poznámka: Po odstránení jedla sa poradové čísla jedál môžu zmeniť.

Implementácia:

Názvy jedál v ponuke budeme v pamäti udržiavať ako zoznam reťazcov: `List<String>`.

Index v zozname môžeme použiť priamo ako jeho relatívnu URI jedla. Implementácia metód služby je

jednoduchá, možno volať priamo metódy zoznamu.

```
@Singleton
@Path("/menu")
public class JedalenResource {
    private List<String> jedla;
    public JedalenResource() {
        jedla = new ArrayList<>();
    }

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getPocet() {
        return "" + jedla.size();
    }

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    public void pridajJedlo(String content) {
        jedla.add(content);
    }

    @GET
    @Path("/{cislo}")
    @Produces(MediaType.TEXT_PLAIN)
    public String getJedlo(@PathParam("cislo") int n) {
        if (n < 0 || n >= jedla.size()) {
            return null;
        }
        return jedla.get(n);
    }

    @DELETE
    @Path("/{cislo}")
    public void odstranJedlo(@PathParam("cislo") int n) {
        if (n < 0 || n >= jedla.size()) {
            return;
        }
        jedla.remove(n);
    }
}
```

Poznámky:

- Ak by mali byť jedlá v ponuke číslované od 1, museli by sme k indexu pripočítať 1, pretože zoznam je indexovaný od 0.
- Pri odstránení položky zo zoznamu sa položky za ňou posunú a tak sa ich relatívne URI znížia, s tým však naša špecifikácia tiež počíta.
- Táto jednoduchá služba nemá metódu PUT na modifikáciu jedál. **Modifikáciu jedla** môžeme urobiť nepriamo, tak že ho odstránime a znovuvytvoríme. Pritom sa však zmení jeho URL. *(Implementácia bez zmeny URL tiež síce možná, pri použití zoznamu však trochu zložitejšia.)*
- Pripomeňme tiež, že POST sa aplikuje vždy na URI kontajnera. Od klienta dostane len reprezentáciu nového zdroja, nie však jeho URI. URI novovytvoreného zdroja (v našom prípade poradové číslo nového jedla) definuje servis, klient sa ju však nedozvie. Preto by bolo dobre keby POST relatívnu URI vrátil.

Implementácia príkladu 1 s metódou POST, ktorá vracia číslo jedla nového, je v projekte [JedalenPost](#).

Príklad 2: REST služba jedálne – varianta PUT

Ponuku jedálne predstavuje opäť koreňový zdroj s URI [menu](#). Zdroj reprezentujúci konkrétne jedlo v ponuke bude mať URI [menu/{nazov}](#), kde nazov je názov jedla. Jeho textová reprezentácia môže teraz poskytovať opis jedla s ďalšími informáciami o ňom (cena, alergény...).

Pre URI ponuky dňa [menu](#) poskytuje služba metódy:

- **GET** pre MIME text/plain: vráti reťazec obsahujúci názvy jedál oddelené medzerou.

Poznámka. Metóda pre POST nie je implementovaná. Namiesto toho sa bude nový zdroj pridávať požiadavkou PUT pre URI nového zdroja – jedla.

Pre URI jedla `menu/nazov` poskytuje služba metódy:

- **GET** pre MIME text/plain: vráti opis jedla s daným názvom.
- **PUT** pre MIME text/plain: zmení opis jedla s daným názvom alebo pridá nové jedlo (*náhrada za POST*)
- **DELETE**: odstráni jedlo s daným názvom.

Poznámka k implementácii:

Na uloženie informácií o jedlách v pamäti môžeme použiť kontajner `Map<String, String>`, kde kľúč bude názov jedla a hodnota jeho opis.

Implementáciu príkladu 2 nájdete v projekte [JedalenPut](#).

8.2 Zdroje so štruktúrovanými dátami

REST služby umožňujú posilať medzi klientom a serverom aj štruktúrované dáta, a to vo formáte XML alebo JSON.

Prácu so štruktúrovanými údajmi si ilustrujeme opäť na REST službe jedálne. Obe varianty implementácie služby sa dajú upraviť tak, že reprezentácia jedla bude štruktúrovaná.

Príklad 3: REST služba jedálne – varianta POST XML

Reprezentácia jedla vo formáte XML bude mať štruktúru:

```
<jedlo>
  <nazov>gulas</nazov>
  <cena>3.5</cena>
</jedlo>
```

Poznámka: Štruktúru XML dokumentov budeme vo výklade zvyčajne opisovať len neformálne, pomocou ilustračných ukážok/príkladov. XML dokumenty, s ktorými pracuje služba, sa môžu líšiť textovým obsahom, príp. aj počtom elementov.

Rovnako ako vo **variante POST** (príklad 1) budú jednotlivé jedlá v ponuke prístupné ako zdroje s URL `menu/n`, kde `n` je poradové číslo jedla v ponuke, pričom jedlá sú číslované za sebou od 0.

Pre URI ponuky `menu` poskytuje služba metódy

- **GET**: Vráti aktuálny počet jedál v menu – pre MIME text/plain.
- **POST**: Pridá nové jedlo do menu.
Akceptuje XML s informáciami o novom jedle – MIME `application/xml`.
Vráti poradové číslo jedla – MIME text/plain.

Pre URI jedla `menu/{n}` poskytuje metódy

- **GET**: Vráti XML s informáciami o jedle - MIME `application/xml`.
- **DELETE**: Odstráni jedlo s poradovým číslom `n`.

Implementácia

Na reprezentáciu informácií o jedle definujeme dátovú triedu `Jedlo`. Technológia JAX-RS umožňuje (prostredníctvom komponentu JAXB) automatickú serializáciu (zápis) a deserializáciu (načítanie) dátových objektov do/z XML alebo JSON formátu. Stačí, ak má dátová trieda anotáciu `@XmlRootElement` a štandardnú java-bean štruktúru.

Pozor. Pom súbore musí obsahovať závislosť na komponente JAXB.

```
@XmlRootElement
public class Jedlo implements Serializable {
    private String nazov;
    private double cena;
```

```
// gettre a settre...
```

Poznámka: Názov koreňového elementu XML dokumentu bude odvodený od názvu triedy, názvy pod-elementov sú odvodené od názvov dátových členov. Toto štandardné mapovanie možno zmeniť. JAXB poskytuje celý rad anotácií, ktorými možno prispôbiť názvy elementov a štruktúru dokumentu vytvoreného serializáciou dátovej triedy. Pozri napr. [JAXB user guide](#).

Implementácia triedy zdroja `JedalenResource` je podobná ako v projekte 1. Stačí nahradiť zoznam reťazcov zoznamom `List<Jedlo>` a upraviť implementáciu metód POST a GET:

- MIME-type treba zmeniť na `application/xml` alebo `application/json`.
- Argument metódy POST a návratová hodnota metódy GET budú typu `Jedlo`.

Implementáciu príkladu 3 nájdete v projekte [JedalenPostPojo](#).

Podobným spôsobom môžeme modifikovať aj variantu služby z príkladu 2, ktorá používa požiadavku PUT na pridávanie, resp. modifikáciu jedál. Pozri cvičenie 8.7.

Implementácia klienta v jazyku Java

Štruktúrované dáta, ktoré posiela REST služba vo formáte XML alebo JSON, môže klient vždy načítať ako textový dokument. Napríklad klient služby z príkladu 3 by mohol na získanie informácií o jedle použiť jednoduchú funkciu:

```
public void getJedlo(int n) {
    String BASE_URI = "http://localhost:8080/JedalenPostPojo/resources";
    Client client = ClientBuilder.newClient();
    WebTarget rootResource = client.target(BASE_URI).path("menu");
    WebTarget subResource = rootResource.path("" + n);
    String result = subResource.request(MediaType.APPLICATION_XML).get(String.class);

    System.out.println("GET returned: " + result);
}
```

Funkcia očakáva odpoveď na GET vo formáte XML: `.request(MediaType.APPLICATION_XML)`, ale odpoveď načíta ako reťazec: `.get(String.class)`.

Samozrejme, ak máme k dispozícii dátovú triedu použitú na strane servera, môžeme ju použiť aj na strane klienta. Volanie požiadavky GET môžeme potom upraviť tak, aby sa odpoveď automaticky načítala do inštancie dátovej triedy.

```
Jedlo result = subResource.request(MediaType.APPLICATION_XML).get(Jedlo.class);
```

Funkciu na pridanie nového jedla môžeme na strane klienta implementovať takto:

```
public String pridajJedlo(Jedlo jedlo) {
    String BASE_URI = "http://localhost:8080/JedalenPostPojo/resources";
    Client client = ClientBuilder.newClient();
    WebTarget rootResource = client.target(BASE_URI).path("menu");
    return webTarget.request(MediaType.TEXT_PLAIN)
        .post(Entity.entity(jedlo, MediaType.APPLICATION_XML), String.class)
}
```

Informácie o jedle posiela klient požiadavkou POST na zdroj menu vo formáte XML:

`.entity(jedlo, MediaType.APPLICATION_XML)`.

Odpoveď očakáva v textovom formáte: `.request(MediaType.TEXT_PLAIN)`.

8.3 Integrácia REST a JPA

Dátové objekty reprezentujúce stav zdroja môžu byť na strane servera persistentne uložené v relačnej databáze. Na prístup k databáze a synchronizáciu ich stavu je vhodné použiť JPA.

V príkladoch budeme používať databázu DerbyDB, ktorá je integrovaná ako súčasť aplikačného servera Glassfish.

Konfigurácia JPA pre web aplikáciu

Databázové web-aplikácie bežiacie na aplikačnom servere môžu využívať **transakčný mechanizmus JTA** poskytovaný aplikačným serverom, ktorý automaticky otvára a ukončuje samostatnú databázovú transakciu pre každú HTTP požiadavku.

Na použitie transakčného mechanizmu JTA potrebné upraviť súbor **persistence.xml**:

```
<persistence-unit name="my_persistence_unit" transaction-type="JTA">
  <jta-data-source>jdbc/sample</jta-data-source>
</persistence-unit>
```

Element **jta-data-source** obsahuje identifikátor konfigurácie s údajmi na pripojenie k databáze. Táto konfigurácia sa však nenachádza priamo v súbore persistence.xml, ale je definovaná v rámci konfigurácie aplikačného servera.

Triedu zdroja vytvoríme ako EJB komponent, čo dosiahneme napríklad anotáciou **@Stateless**. Referenciu na manažér entít môžeme potom do triedy zdroja injektovať anotáciou **@PersistenceContext**.

```
import jakarta.ejb.Stateless;
@Stateless
@Path("menu")
public class JedalenResource {
    @PersistenceContext(unitName = "my_persistence_unit")
    private EntityManager em;
    // ...
}
```

S využitím manažéra entít môžu metódy služby synchronizovať stav zdrojov okamžite s databázou.

Príklad 4: REST služba jedálne – varianta POST s ukladaním v databáze

Implementácia:

Informácie o jedlách budú persistentne uložené v databázovej tabuľke JEDLO. Dátová trieda **Jedlo** je teda entitnou triedou. Namiesto indexu v zozname bude teraz relatívnou URI jedla hodnota kľúča typu **long**. Keďže na vytváranie nových zdrojov použijeme v tejto variante metódu POST, kľúč predstavujúci relatívnu URI nového zdroja je definovaný serverom. Môže byť teda generovaný priamo databázou.

```
@Entity
@XmlRootElement
public class Jedlo implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String nazov;
    private double cena;
    // ...
}
```

Implementácia metód zdroja je priamočiara. Napríklad GET metódu, ktorá vracia informácie o konkrétnom jedle, môžeme implementovať jednoduchým volaním metódy **find** manažéra entít.

```
@GET
@Path("/{cislo}")
@Produces(MediaType.APPLICATION_XML)
public Jedlo getJedlo(@PathParam("cislo") long id) {
    return em.find(Jedlo.class, id);
}
```

Implementáciu príkladu 4 nájdete v projekte [JedalenPostDB](#).

Cvičenia

Cvičenie 8.1

Implementujte java klientov pre varianty REST služby jedálne z príkladov 1,2 a 3, ktorými otestujete pridávanie a odstraňovanie jedál.

Cvičenie 8.2 REST služba jedálne – použitie formátu JSON

Upravte príklad 2 (projekt [JedalenPut](#)) tak, aby požiadavka GET pre koreňový zdroj menu vrátila namiesto jedného reťazca zoznam s názvami jedál vo formáte JSON.

Návod: Návratovú hodnotu metódy zdroja zmeňte na `java.util.list` a typ média v anotácii na `APPLICATION_JSON`. Všimnite si, že zoznam bude automaticky serializovaný do formátu JSON.

Poznámka. Skúste, či je možné vrátiť zoznam reťazcov aj vo formáte XML .

Cvičenie 8.3 REST služba jedálne – GET vracajúca zoznam objektov

V projekte [JedalenPostPojo](#) doplňte metódu pre požiadavku GET na koreňový zdroj, ktorá vráti zoznam všetkých jedál vo formáte:

- a) JSON
- b) XML

Poznámka: Služba bude teda pre kolekciu `menu` poskytovať dve GET metódy. To je možné, keďže vracajú reprezentáciu zdroja v rôznych formátoch. Pomocou prehliadača otestujte obe metódy.

V prípade b) si všimnite názov koreňového XML-elementu v odpovedi.

Cvičenie 8.4 REST služba jedálne – doplnenie metódy PUT do varianty POST XML

Do projektu [JedalenPostPojo](#) doplňte implementáciu metódy pre požiadavku PUT, ktorá umožní klientovi modifikovať cenu, prípadne názov jedla v ponuke.

Pozor: Na rozdiel od POST sa požiadavka PUT posiela na URI konkrétneho jedla a zmení jeho stav (cenu,...). Nevytvára nový zdroj. Implementujte ju preto tak, aby nezmenila relatívnu URI zdroja.

Cvičenie 8.5 REST služba jedálne – doplnenie metódy PUT do varianty POST s ukladáním v databáze

Do projektu [JedalenPostDB](#) doplňte implementáciu metódy pre požiadavku PUT, ktorá umožní klientovi modifikovať cenu, prípadne názov jedla v ponuke.

Cvičenie 8.6 REST služba jedálne – modifikácia varianty POST s ukladáním v databáze

Do projektu [JedalenPostDB](#) doplňte implementáciu GET metódy pre menu, ktorá vráti zoznam číselných kľúčov (teda relatívnych URI) jedál aktuálne v menu vo formáte JSON.

Cvičenie 8.7 REST služba jedálne – varianta PUT XML

Reimplementujte príklad 2 (projekt [JedalenPut](#)) tak, aby informácie o jedle boli v pamäti reprezentované ako objekty triedy `Jedlo` a posielané vo formáte XML.

```
public class Jedlo {  
    private String popis;  
    private double cena;  
    private List<Short> alergen;
```

Všimnite si, akým spôsobom bude zoznam alergénov serializovaný do XML.

Cvičenie 8.8 REST služba jedálne – varianta PUT s ukladáním v databáze

Reimplementujte projekt z cvičenia 8.7 tak, aby informácie o jedlách boli ukladané v databáze.

Pozor: Do triedy `Jedlo` je treba doplniť kľúč – názov jedla:

```
public class Jedlo {  
    @Id  
    private String nazov;  
    private String popis;  
    private double cena;  
    private List<Short> alergen;
```

Cvičenie 8.9 Overenie funkčnosti REST služby z cvičenia 8.8

Pri posielaní požiadavky PUT na službu jedálne s ukladáním v databáze (cvičenie 8.8), zadáva klient názov jedla ako relatívnu URI, ale môže ho zadať v XML-elemente `nazov`, keďže `nazov` je kľúčový dátový člen entitnej triedy `Jedlo`.

Implementujte klientskú aplikáciu, ktorá preverí spracovanie PUT požiadavky, keď klient

- a) v XML-elemente `nazov` zadá iný názov pre jedlo ako v URI,
- b) XML-element `nazov` úplne vynechá.

Aplikácia najprv pošle PUT požiadavku na vytvorenie nového jedla a potom si GET požiadavkou preverí, či bolo nové jedlo vytvorené, má požadované URI a stav.

Odporúčanie:

Aby sa vyhlo problémom spôsobeným klientom, ktorý zadá iný názov jedla do URI a do obsahu PUT správy, mala by metóda služby použiť ako kľúč v databáze hodnotu uvedenú v URI. Hodnotu, ktorú klient zadá do XML-elementu, môže metóda ignorovať, resp. klient ju vôbec nemusí poslať.

To môžeme dosiahnuť anotáciou `@XmlTransient` dátového člena `nazov`.

Cvičenie 8.10 REST služba jedálne s menu pre celý týždeň

Menu s ponukou pre jeden deň bude prístupné ako zdroj s URI `menu/{den}`, kde `den` je meno dňa v týždni. Platné mená dní sú reťazce: `pondelok`, `utorok`, `streda`, `stvrtok`, `piatok`.

Zdroj reprezentujúci konkrétne jedlo v ponuke dňa bude mať URI `menu/{den}/{nazov}`, kde `nazov` je názov jedla. Jeho reprezentácia bude mať formát XML so štruktúrou ako v cvičení 8.7.

Pre URI ponuky dňa `menu/{den}` poskytuje služba metódu:

- GET – vráti zoznam názvov jedál v ponuke dňa vo formáte JSON.

Pre URI jedla `menu/{den}/{nazov}` poskytuje služba metódy:

- GET pre MIME APPLICATION_XML – vráti reprezentáciu jedla vo formáte XML.
- PUT pre MIME APPLICATION_XML – aktualizuje informácie o jedle.
- DELETE – odstráni jedlo s daným názvom.

Metóda GET pre koreňové URI `menu` umožňuje tiež zistiť, ktorý deň je jedlo v ponuke. Názov jedla zadá klient ako parameter požiadavky (`@QueryParam("nazov")`). Zoznam dní, keď je jedlo s daným názvom v ponuke, vráti metóda vo formáte JSON.

- A. Implementujte službu tak, že všetky informácie budú udržiavané len v pamäti, (použite vhodné dátové štruktúry).
Službu inicializujte tak, že pri jej spustení bude už menu pre každý deň obsahovať pizzu za 6.50.
- B. Pomocou vhodného klienta službu otestujte: Metódou PUT zmeňte cenu pizze v pondelkovom menu a pozrite sa, či sa pri tom zmenila cena pizze aj v ostatných dňoch.

Cvičenie 8.11 REST služba jedálne s menu pre celý týždeň s ukladaním v databáze

Reimplementujte REST službu z cvičenia 8.10 tak, že informácie o menu pre celý týždeň budú uložené v databáze.

Implementáciu služby otestujte klientom rovnako ako v cvičení 8.10. Porovnajte, či sa obe implementácie služby správajú rovnako pri zmene ceny pizze.

Cvičenie 8.12 REST služba jedálne s menu pre celý týždeň s ukladaním v databáze – varianta POST

Implementujte službu jedálne s menu pre celý týždeň s ukladaním v databáze, pričom jedlá budú do menu vkladané pomocou metódy POST. (Teda ich identifikátor bude kľúč generovaný databázou, podobne ako v projekte `JedalenPostDB`.)

Pre URI `menu/{den}/{id}` implementujte tiež metódu PUT na modifikáciu informácií o jedle.

Implementáciu služby otestujte klientom rovnako ako v cvičeniach 8.10 a 8.11. Porovnajte ich správanie pri zmene ceny pizze.